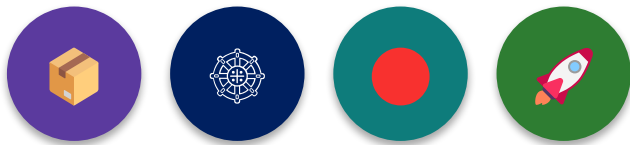


"The cluster is the runtime — if it is not in the image, it does not exist."

SUPPLEMENTARY · MODULE 10

Containers and Red Hat OpenShift

Package and run services on OpenShift: images, probes, ConfigMaps, routes, and the platform primitives that replace localhost assumptions.



CONTAINERS VS. VIRTUAL MACHINES

A VM packages an entire OS plus the app; a container packages just the app and its dependencies, sharing the host kernel underneath.

VM VS. CONTAINER

AREA	VIRTUAL MACHINE	CONTAINER
Includes full OS	Yes	No
Startup time	Slower	Faster
Resource usage	Heavier	Lighter
Packaging	OS plus app	App plus dependencies

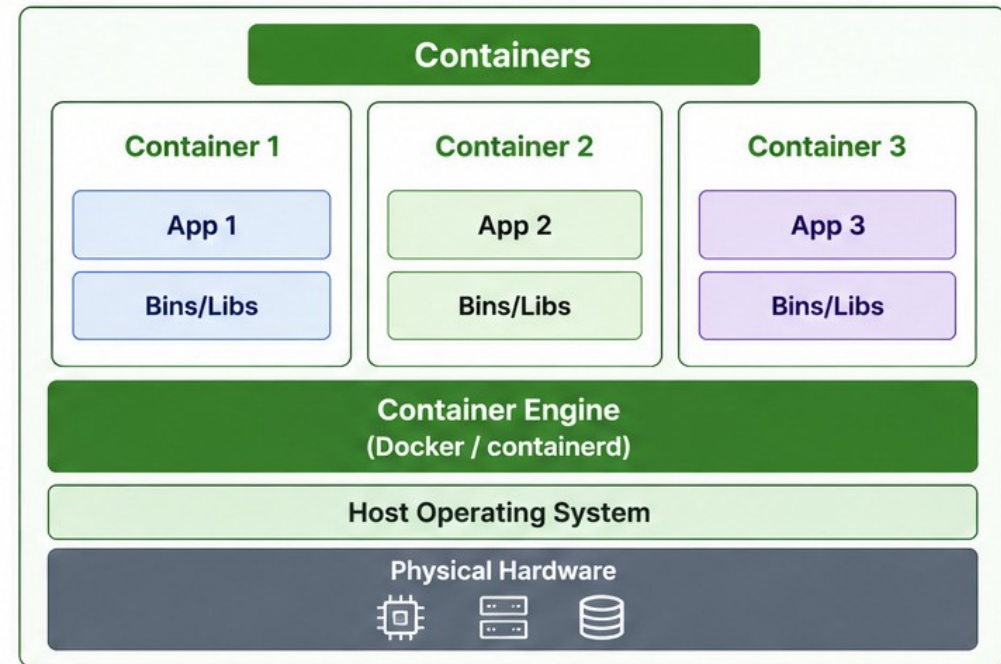
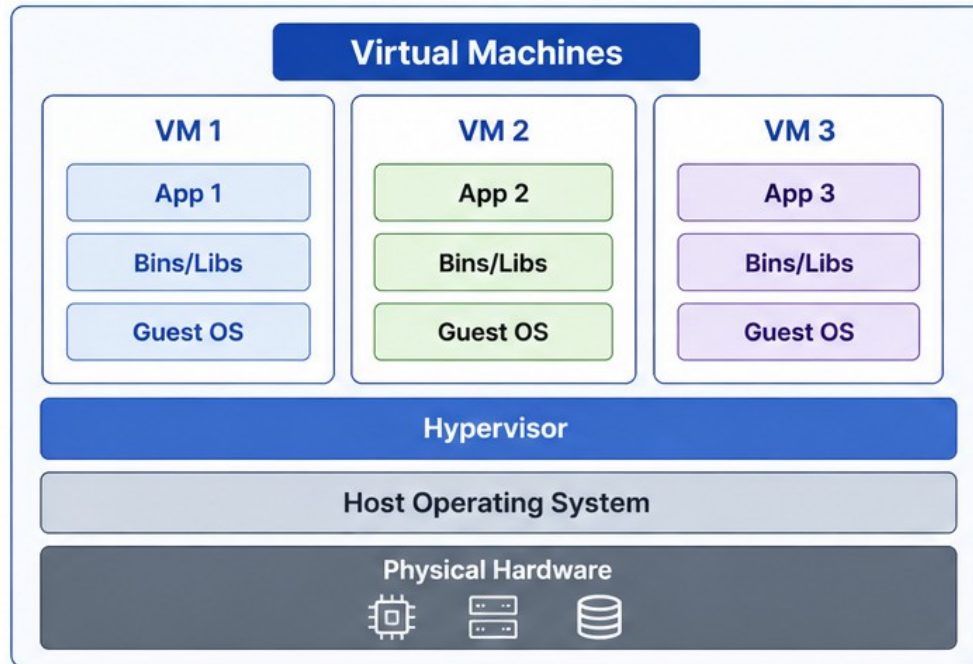


IMPORTANT CLARIFICATION

- Containers are not automatically secure or production-ready
- Still require secure base images, vulnerability scanning, secrets management, and resource limits

KEY TAKEAWAY A VM packages an entire OS plus the app; a container packages just the app and its dependencies, sharing the host kernel underneath.

Containers vs Virtual Machines



Aspect	Virtual Machines	Containers
 Architecture	Each VM runs a full Guest OS	Containers share the Host OS kernel
 Startup Time	Minutes	Milliseconds to seconds
 Resource Overhead	High (Guest OS per VM)	Low (No Guest OS)
 Density	Lower (Fewer VMs per host)	Higher (More containers per host)
 Isolation	Strong (Full OS isolation)	Process-level isolation
 Cost Efficiency	Higher infrastructure cost	Lower infrastructure cost
 Management	Manage OS, patches, VMs	Manage containers and images
 Portability	Lower (Tied to OS and hypervisor)	Higher (Consistent across environments)
 Use Cases	Legacy apps, different OS, strong isolation	Microservices, cloud-native apps, CI/CD

LINUX CONTAINER FUNDAMENTALS · IMAGES, CONTAINERS, AND REGISTRIES · CONTAINER IMAGE LAYERS



THE TWO KERNEL FEATURES BEHIND CONTAINERS

- **Namespaces:** isolate a process's view of PIDs, network, mounts, users, and hostname
- **Control groups (cgroups):** limit and account for CPU, memory, and I/O usage per process group
- A container runtime (containerd, CRI-O) orchestrates these kernel primitives



THE THREE CONCEPTS

- **Image:** a packaged template -- `account-service:1.0.0`
- **Container:** a running instance of that image
- **Registry:** stores and distributes images so platforms like OpenShift can pull them



HOW LAYERING WORKS

- Each instruction (FROM, COPY, RUN) creates a new, cacheable image layer
- Unchanged layers are reused across builds, unchanged parts don't need to rebuild
- Poor layer design creates large images or slow, uncacheable builds

KEY TAKEAWAY Under the hood, a container is just a regular Linux process wrapped in kernel namespaces and control groups, not a separate virtualized OS. • `account-service:1.0.0` the image becomes a running Account Service process the moment it's started as a container. • Good layer ordering (least-changing instructions first) lets Docker reuse cached layers instead of rebuilding the entire image every time.

CONTAINER RUNTIME CONCEPTS · WRITING A DOCKERFILE FOR SPRING BOOT · DOCKERFILE INSTRUCTIONS AND BUILD CONTEXT



RUNTIME CONCEPTS

- **Container runtime:** the software that creates and manages the container lifecycle on a node
- **CRI (Container Runtime Interface):** the standard API Kubernetes uses to talk to any compliant runtime
- OpenShift uses CRI-O by default -- a lightweight, Kubernetes-native runtime



COMMON DOCKERFILE INSTRUCTIONS

- FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, USER, ENTRYPOINT, CMD



BASIC SPRING BOOT DOCKERFILE

- FROM eclipse-temurin:17-jre WORKDIR /app
- COPY target/account-service.jar app.jar EXPOSE 8080
- ENTRYPOINT ["java", "-jar", "app.jar"]



BUILD CONTEXT DISCIPLINE

- The build context is the file set sent to the image builder
- Use .dockerignore to exclude .git, .idea, *.log, and other unneeded files

KEY TAKEAWAY containerd and CRI-O implement the Container Runtime Interface that Kubernetes and OpenShift use to run pods on every node.

- FROM eclipse-temurin:17-jre, COPY the JAR, EXPOSE 8080, ENTRYPOINT java -jar app.jar -- the minimum viable Spring Boot Dockerfile.
- Without a .dockerignore, .git folders, local credentials, and IDE metadata can silently end up inside the build context and even the image.

MULTI-STAGE CONTAINER BUILDS · LAYERED SPRING BOOT IMAGES



MULTI-STAGE CONCEPT

- FROM maven:3.9-eclipse-temurin-17 AS build -> mvn clean package
- FROM eclipse-temurin:17-jre -> COPY --from=build ... app.jar
- The final image contains only the runtime and the compiled JAR, nothing from the build stage



WHY LAYERED JARS HELP

- spring-boot:build-image or layertools extracts dependencies, resources, and application classes into separate layers
- Dependencies rarely change -- that layer stays cached across most builds
- Only the application-classes layer typically needs to be rebuilt and re-pushed

KEY TAKEAWAY The Maven build tools, source code, and intermediate files never make it into the final image -- only the compiled JAR does. • Since dependencies change far less often than your own code, layering means most rebuilds only need to push one small, changed layer.

Layered JARs and Image Optimization

Layered JARs (Spring Boot)

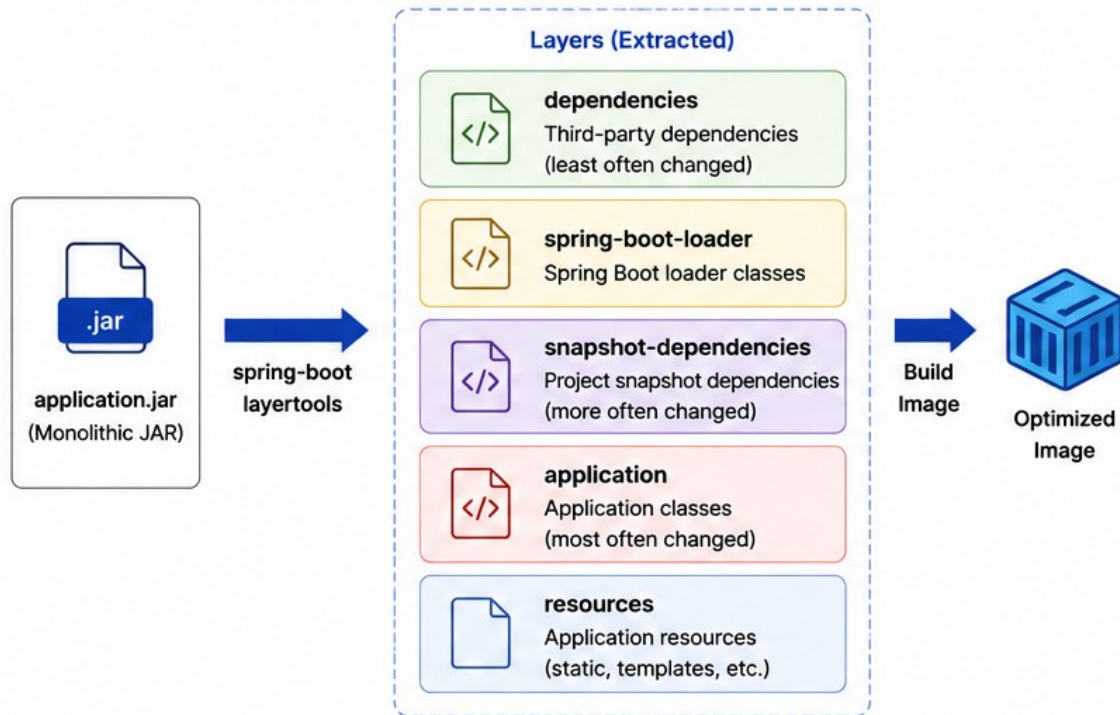


Image Optimization Best Practices



Use Layered JARs

Maximize layer caching by separating dependencies from application code.



Use Minimal Base Image

Use slim or distroless images to reduce size and surface area.



Reduce Layers

Combine RUN commands and avoid unnecessary layers.



Remove Unnecessary Files

Exclude build tools, docs, tests, and cache.



Run as Non-Root User

Improve security by avoiding root user.



Optimize for Size and Speed

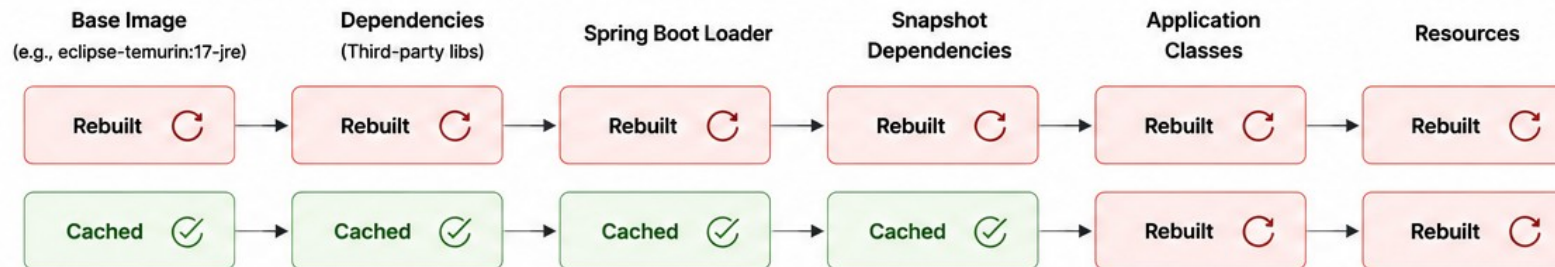
Smaller images pull faster, start faster, and use fewer resources.

Layer Caching Benefit

When application code changes:

Without Layered JAR
(Monolithic JAR)

With Layered JAR
(Layered JAR)



Reused (from cache)



Rebuilt (changed layer)

Result: Faster builds, smaller images, efficient caching

BUILDPACKS AND CLOUD-NATIVE BUILDPACKS · CONTAINER IMAGE TAGGING STRATEGIES · CONTAINER REGISTRIES



WHY BUILDPACKS HELP

- Automatically detect the language/framework and apply best-practice image construction
- No Dockerfile to write or maintain; consistent, reproducible builds across teams
- `mvn spring-boot:build-image` is the common Spring Boot entry point



COMMON REGISTRIES

- Red Hat Quay, OpenShift internal registry, Docker Hub, GitHub Container Registry, cloud provider registries



GOOD TAGGING EXAMPLES

- **Semantic version:** 1.4.2
- **Git commit SHA:** a1b2c3d
- **Build number:** build-105
- Avoid relying only on mutable tags like `latest` for traceable deployments



REGISTRY SECURITY

- Authentication, authorization, image scanning, audit logging, image signing
- Image pull secrets give OpenShift credentials to pull from authenticated registries

KEY TAKEAWAY `spring-boot:build-image` uses Paketo buildpacks under the hood, automatically detecting Java and producing a production-ready, layered image. • Semantic versions, Git commit SHAs, and build numbers all answer 'what exactly is deployed?' -- `latest` never does. • Authentication, image scanning, and image pull secrets turn a registry from a simple file store into an enterprise-grade delivery gate.

IMAGE IMMUTABILITY · CONTAINER CONFIGURATION AND ENVIRONMENT VARIABLES · CONTAINER NETWORKING



WHY IMMUTABILITY MATTERS

- An image tag that can be overwritten breaks the guarantee that a deployment is reproducible
- Immutable, versioned tags (1.0.0, 1.0.1) make rollbacks and audits reliable
- Pin to a specific digest or version tag in production, never a mutable one



ENVIRONMENT VARIABLES IN PRACTICE

- `docker run -e SPRING_PROFILES_ACTIVE=prod account-service:1.0.0`
- Supports the build-once, configure-per-environment model
- Spring Boot reads environment variables directly into its configuration properties



PORT MAPPING EXAMPLE

- Spring Boot listens on port 8080 inside the container
- `docker run -p 8081:8080 account-service:1.0.0` maps that to host port 8081
- The service becomes reachable locally at `http://localhost:8081`

KEY TAKEAWAY Immutable, versioned tags guarantee that the same tag always deploys the exact same bits, in every environment, every time.

- `docker run -e SPRING_PROFILES_ACTIVE=prod` passes configuration in without ever touching or rebuilding the image.
- `docker run -p 8081:8080` exposes a service listening on container port 8080 to the outside world on host port 8081.

Registries, Tagging, and Immutability

1. Registries



Container Registry

A registry stores, manages, and distributes container images.

Types of Registries



Public Registries

e.g., Docker Hub, GHCR



Private Registries

e.g., Harbor, ECR, ACR



Local Registries

e.g., Nexus, Artifactory

Common Operations



Push

Upload images



Pull

Download images



Search

Find images



Delete

Remove images

2. Tagging

Tags are human-friendly references to images. They make images easy to identify and manage.

Tag Format

`<registry>/<repository>:<tag>`

Example:

`registry.example.com/payments-service:1.2.3`

Tag vs. Digest

	Tag	Digest (Immutable)
Human Readable	✓	✗
Mutable	✓	✗
Unique Identifier	✗	✓
Example	1.2.3	sha256:3ad...9f2c

Common Tagging Strategies



Semantic Versioning

1.2.3, 1.2.4, 2.0.0



Date Based

2025-05-09



Git Commit SHA

a1b2c3d4



Environment

dev, test, staging, prod, latest

3. Immutability

Immutability ensures images cannot be changed once published. It improves security, reliability, and traceability.

Mutable Tag (Not Recommended)

Step 1

Push image with tag
payments-service:1.0.0



Image A
(sha256:aaa...)

Step 2

Push new image with same tag
payments-service:1.0.0



Image B
(sha256:bbb...)



Tag 1.0.0 now points to a different image (overwritten).
Risk: Non-deterministic deployments, hard to reproduce.

Immutable Tag / Digest (Recommended)

Push image with tag
payments-service:1.0.0



Image A
(sha256:aaa...)

New Version

Push new image with tag
payments-service:1.0.1



Image B
(sha256:bbb...)



Each tag points to a unique image.
Digest (sha256:...) is always immutable.

Enforce Immutability



Enable immutable tags
in your registry



Prevent tag overwrites
(e.g., disallow pushing to
existing tags)



Use digests in deployments
for guaranteed immutability



Secure policies (RBAC)
and audit logging

Benefits



Security

Prevents tampering
and supply chain risks



Reliability

Consistent, repeatable
deployments



Traceability

Easy to track what was
deployed and when



Rollbacks

Predictable and safe
rollbacks

CONTAINER STORAGE · CONTAINER RESOURCE LIMITS · RUNNING AS A NON-ROOT USER



EPHEMERAL VS. PERSISTENT STORAGE

- **Container filesystem:** temporary, lost when the container is removed or replaced
- **Volumes:** external storage mounted into the container, surviving restarts and replacements
- Stateless microservices should avoid depending on local container storage entirely



WHY RESOURCE LIMITS MATTER

- Without limits, one runaway container can exhaust node CPU or memory
- Requests reserve a guaranteed minimum; limits cap the maximum
- Both should be set deliberately based on real observed workload



WHY NON-ROOT MATTERS

- Containers should avoid running as root unless explicitly required
- OpenShift enforces security constraints expecting restricted, non-root permissions
- Test images locally as non-root before assuming they'll work on the platform

KEY TAKEAWAY Anything written inside a container's own filesystem disappears when it's replaced; only mounted volumes persist across restarts. • CPU and memory requests and limits protect both the container's own stability and the shared cluster it runs on. • A USER instruction in the Dockerfile, matched to OpenShift's security context constraints, avoids surprise permission failures at deploy time.

Secure Containerfiles and Non-Root Execution



Secure Containerfile Best Practices

```
1 FROM eclipse-temurin:17-jre-alpine
2
3 # Create a non-root user
4 RUN addgroup -S appgroup \
5     && adduser -S appuser -G appgroup
6
7 WORKDIR /app
8
9 COPY --chown=appuser:appgroup app.jar ./app.jar
10
11 USER appuser
12
13 ENTRYPOINT ["java", "-jar", "app.jar"]
```



Use minimal, trusted base images



Create a dedicated non-root user



Set working directory



Copy files with correct ownership



Switch to non-root user



Define entrypoint



Non-Root Execution Benefits



Improved Security

Limits the impact of a container escape or application compromise.



Principle of Least Privilege

Runs the application with only required permissions.



Reduces Risk

Prevents accidental or malicious changes to the container and host system.



Compliance Friendly

Meets security and compliance requirements (e.g., CIS, NIST, PCI DSS).



Better Isolation

Non-root processes are more isolated from the underlying system.



How Non-Root Execution Works



1. Build Image

Image includes a non-root user and proper file ownership.



2. Run Container

Container starts using the specified non-root user.



3. Process Runs

Application runs with limited privileges inside the container.



4. Secure by Default

Even if compromised, the attacker has limited access.



Always run containers as non-root users to build secure and resilient applications.



Additional Security Tips

- ✓ Keep base images up-to-date and scan for vulnerabilities.
- ✓ Use .dockerignore to exclude unnecessary files.
- ✓ Avoid installing unnecessary packages.
- ✓ Drop Linux capabilities if not needed.
- ✓ Set read-only file system when possible (readOnlyRootFilesystem).
- ✓ Use HEALTHCHECK to enable reliable monitoring.
- ✓ Scan images in CI/CD pipeline before deployment.

CONTAINER HEALTH CHECKS · REDUCING IMAGE SIZE AND ATTACK SURFACE · CONTAINER SECURITY BEST PRACTICES



CONTAINER HEALTH CHECK BASICS

- A HEALTHCHECK instruction defines a command the runtime periodically executes
- A failing health check can trigger a container restart independent of the orchestration platform
- In OpenShift, this is complemented (not replaced) by liveness/readiness/startup probes



STRATEGIES TO SHRINK IMAGES

- Use minimal JRE (not JDK) base images for runtime
- Use multi-stage builds so build tools never reach the final image
- Avoid unnecessary package installs and shell utilities in production images



CONTAINER SECURITY CHECKLIST

- Use minimal, approved base images and scan them for vulnerabilities
- Run as non-root; avoid storing secrets inside images
- Pin base image versions per enterprise policy; keep dependencies minimal

KEY TAKEAWAY HEALTHCHECK in a Dockerfile is the container-runtime-level companion to the OpenShift-level liveness and readiness probes covered later. • Minimal JRE base images, multi-stage builds, and removing unnecessary tools all shrink both the download size and the attack surface. • Security isn't one setting -- it's a checklist applied consistently across every image a team builds.

SUPPLY-CHAIN SECURITY AND SBOMS · TESTING CONTAINERS LOCALLY



WHY SBOMS MATTER

- An SBOM inventories every dependency and library baked into an image
- **Enables fast vulnerability response:** search the SBOM instead of re-scanning blindly
- Increasingly a compliance and enterprise procurement requirement



LOCAL TEST LOOP

1. `docker build -t account-service:1.0.0 .`
2. `docker run -p 8081:8080 account-service:1.0.0`
3. `curl http://localhost:8081/actuator/health` to verify it started correctly

KEY TAKEAWAY A Software Bill of Materials turns 'is this image affected by CVE-XXXX' from a guess into a searchable answer. • `docker build`, `docker run -p`, and `curl` the health endpoint form the minimum local test loop before pushing to a registry.



CONTAINERIZATION

Container Troubleshooting Techniques



Identify, diagnose, and resolve container issues quickly to keep your services healthy, reliable, and performant.

COMMON ISSUES



Container won't start
Misconfiguration, missing dependencies, or errors.



Crashes / OOM
Out of memory, high CPU, or resource limits.



Networking problems
DNS resolution, port mapping, or connectivity issues.



Volume / storage
Permission issues, missing mounts, or full disk.



Permission denied
Running as non-root, file system, or capability issues.



Slow performance
High latency, resource contention, or inefficient app.

TROUBLESHOOTING WORKFLOW



1. Identify
What is the symptom?



2. Inspect
Check status, logs, and events.



3. Analyze
Find the root cause.



4. Fix
Apply the fix or config change.



5. Verify
Confirm the issue is resolved.

ESSENTIAL DOCKER COMMANDS

Purpose	Command	Example
List containers	<code>docker ps -a</code>	<code>docker ps -a</code>
Inspect container	<code>docker inspect <name/id></code>	<code>docker inspect my-app</code>
View logs	<code>docker logs [options] <name></code>	<code>docker logs -f my-app</code>
Execute in container	<code>docker exec -it <name> <shell></code>	<code>docker exec -it my-app sh</code>
View resource usage	<code>docker stats</code>	<code>docker stats my-app</code>
Check ports	<code>docker port <name></code>	<code>docker port my-app</code>
View events	<code>docker events</code>	<code>docker events --since 10m</code>
Remove container	<code>docker rm <name></code>	<code>docker rm my-app</code>

DEEP-DIVE TECHNIQUES



Check Container Health
`docker inspect` to view Health status.



Resource Analysis
Use `docker stats` or cAdvisor.



Network Diagnostics
Test DNS, ping, curl, netstat inside the container.



File System Inspection
Check mounts, disk usage, and permissions.



Process Inspection
Check running processes and resource consumption.



Reproduce with Minimal Setup
Run with minimal config to isolate the issue.

LOGS & INSPECTION EXAMPLE

```
# View running containers
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  STATUS    PORTS          NAMES
a1b2c3d4e5f6   my-app:1.0  "java -jar app.jar"      Up 2m     0.0.0.0:8080->8080/tcp   my-app

# Follow logs
$ docker logs -f my-app

# Inspect container
$ docker inspect my-app | less

# Execute shell
$ docker exec -it my-app sh

# Check resource usage
$ docker stats my-app
```

BEST PRACTICES



Use health checks
Detect and recover issues automatically.



Log to stdout/stderr
Centralize logs and monitor effectively.



Set resource limits
Prevent resource exhaustion.



Keep images minimal
Fewer layers, fewer moving parts.



Test in environments
Match production as closely as possible.



Monitor continuously
Alert early and act proactively.

KUBERNETES AND OPENSIFT OVERVIEW · KUBERNETES VS. RED HAT OPENSIFT · OPENSIFT ARCHITECTURE



WHAT OPENSIFT PROVIDES

- Container orchestration and deployment automation built on Kubernetes
- Networking, routing, security controls, image management, monitoring
- **Developer workflows:** web console, oc CLI, image streams, builds



CORE OPENSIFT CONCEPTS

- Cluster, node, pod, deployment, service, route, project
- ConfigMap, secret, image stream, operator



WHAT OPENSIFT ADDS ON TOP OF KUBERNETES

- Security Context Constraints (SCCs) enforce non-root and least-privilege by default
- Routes (vs. plain Ingress), Image Streams, and BuildConfigs for developer workflows
- An integrated web console, registry, and CI/CD-friendly build pipeline

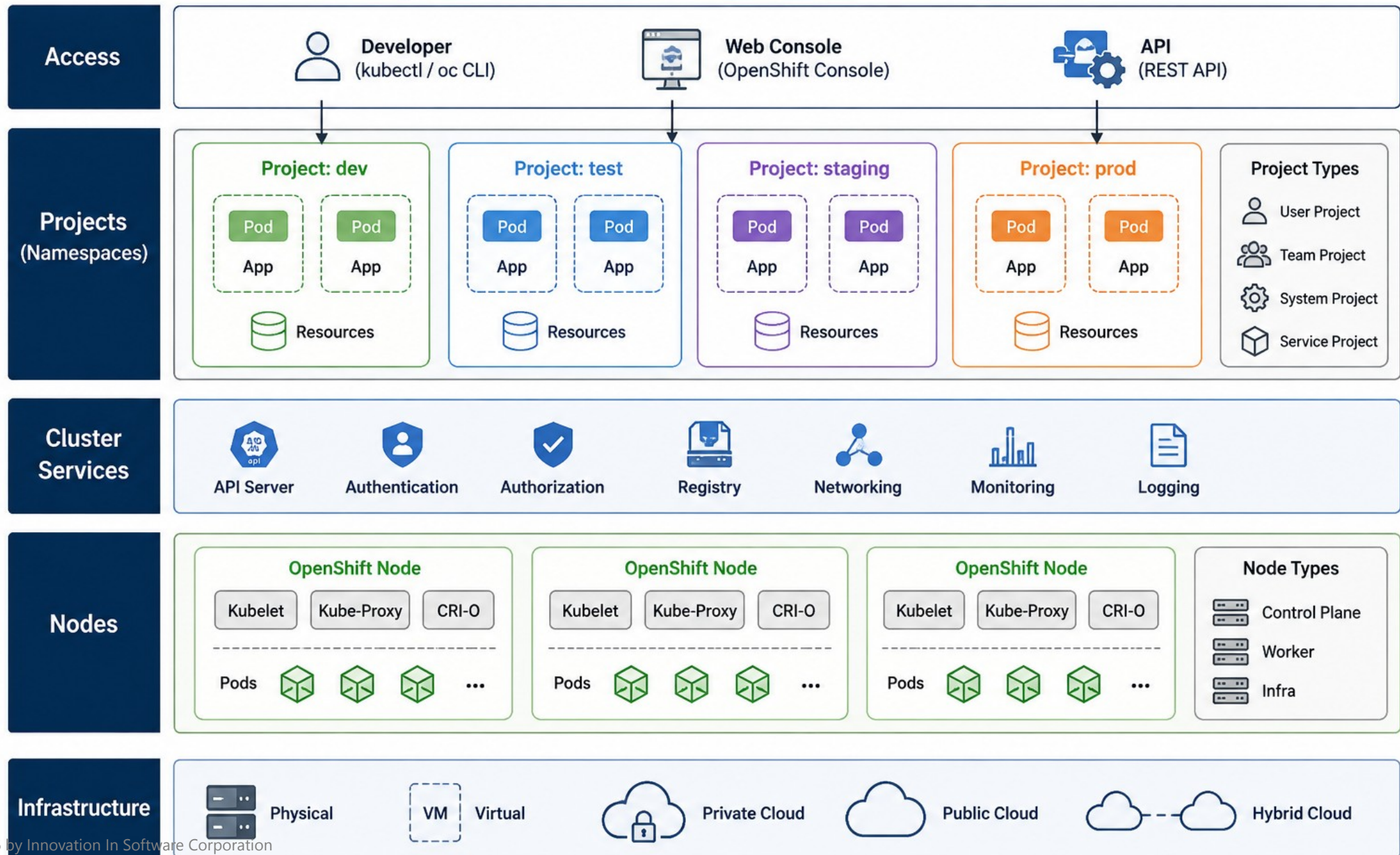


BANKING EXAMPLE

- Assigned OpenShift project containing Account Service and Transaction Service deployments, each with pods, a service, and a route

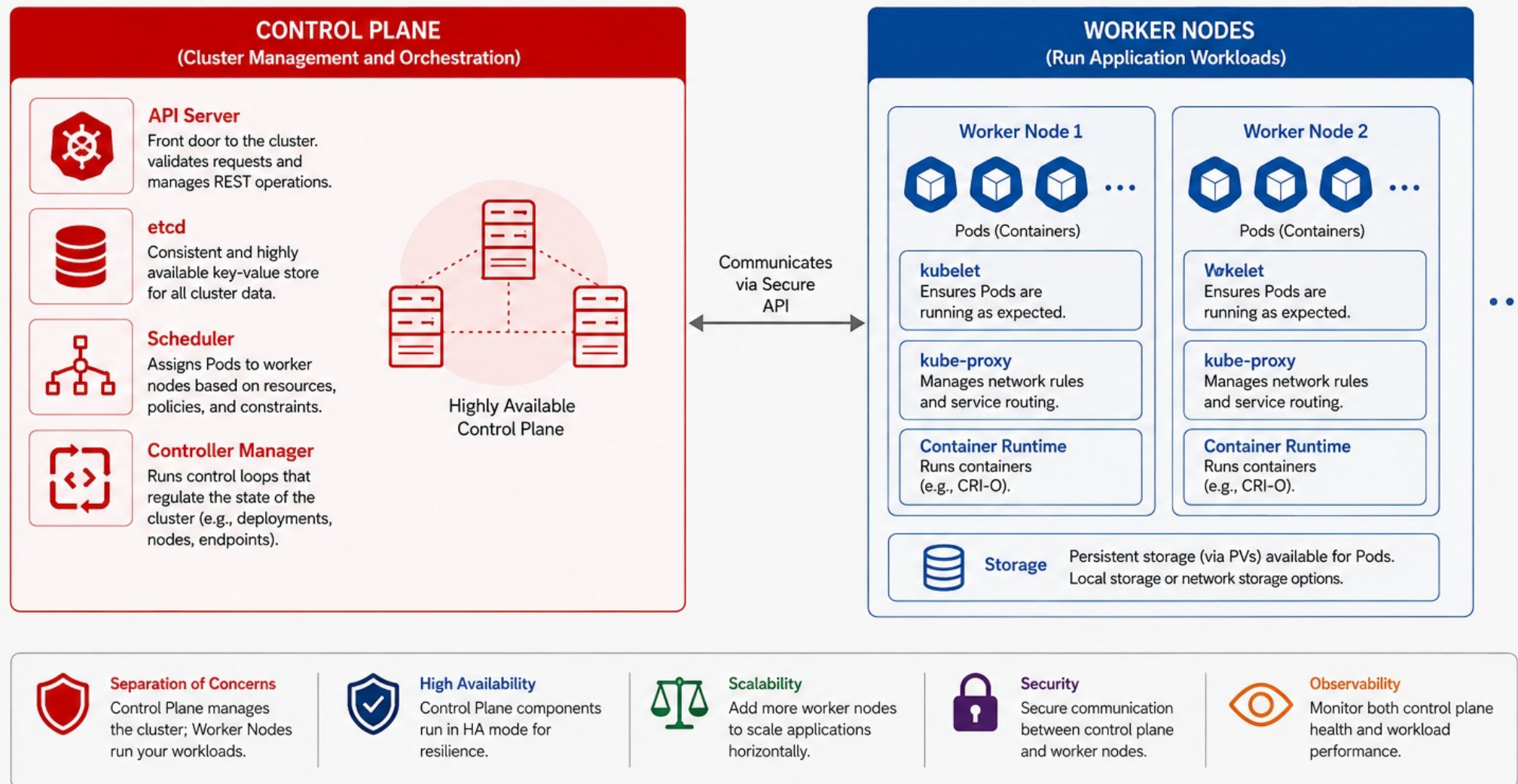
KEY TAKEAWAY Container orchestration, networking, routing, security, image management, and developer workflows all come bundled with OpenShift. • Vanilla Kubernetes gives you the primitives; OpenShift adds Security Context Constraints, Routes, Image Streams, and BuildConfigs on top. • Pods run on worker nodes, deployments manage pods, services provide stable networking, and routes expose selected services externally.

OpenShift Architecture and Projects



CONTROL PLANE AND WORKER NODES

OpenShift cluster architecture is built on a clear separation of responsibilities between the Control Plane and the Worker Nodes.



OPENSIFT PROJECTS AND NAMESPACES · PODS AND REPLICASETS · DEPLOYMENTS AND DEPLOYMENTCONFIGS

WHY PROJECTS MATTER

- Separate development, test, staging, and production environments
- Support team ownership, access control, and resource quotas
- `oc new-project`, `oc project`, and `oc get` all are the core commands

WHAT A DEPLOYMENT MANAGES

- Which container image to run, how many replicas, environment variables, rollout strategy
- `oc rollout status` / `history` / `undo` manage and inspect rollouts

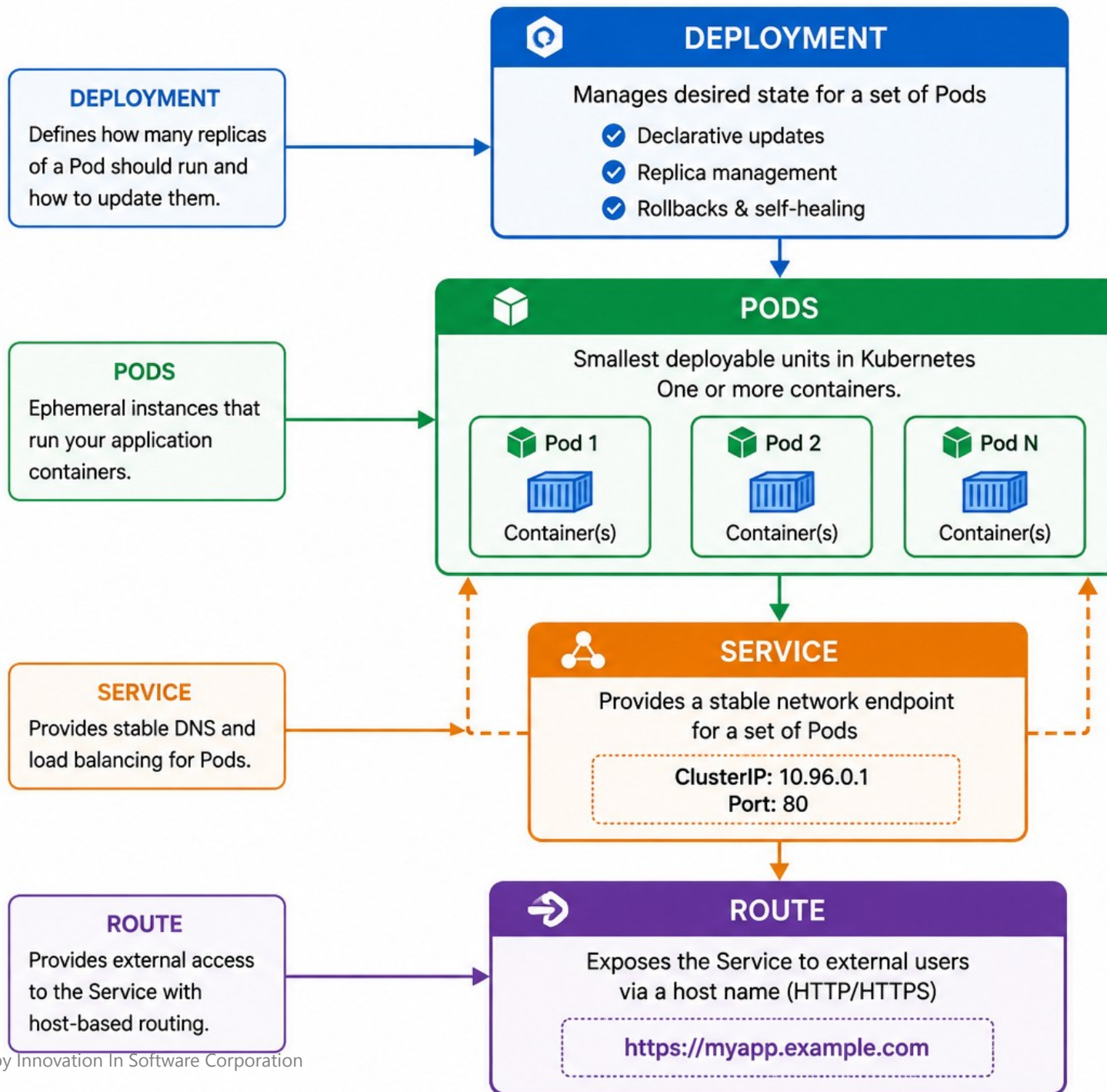
PODS AND REPLICASETS

- A pod usually contains one application container, though it can contain more
- A ReplicaSet ensures a specified number of pod replicas are running at any given time
- Deployments manage ReplicaSets, which in turn manage pods

DEPLOYMENT VS. DEPLOYMENTCONFIG

- **Deployment:** Kubernetes-native, portable, now the recommended default
- **DeploymentConfig:** OpenShift-specific, older, with some OpenShift-only trigger behaviors

KEY TAKEAWAY banking-dev, banking-test, banking-staging, and banking-prod each isolate the same services with different configuration and access rules. • If a pod fails, its ReplicaSet notices and creates a replacement automatically -- no manual intervention required. • Kubernetes-native Deployments are now preferred over OpenShift's older DeploymentConfig for most new Spring Boot workloads.



SERVICES AND INTERNAL NETWORKING · ROUTES AND EXTERNAL ACCESS · CONFIGMAPS AND SECRETS



HOW SERVICES WORK

- A Service is a stable virtual IP and DNS name that provides reliable access to a set of Pods
- Services use label selectors to route traffic to matching Pods via endpoints
- **Types:** ClusterIP (default, internal-only), NodePort, LoadBalancer, ExternalName



SERVICE DISCOVERY

- Services register automatically in DNS as `<service-name>.<namespace>.svc.cluster.local`
- Pod-to-pod communication uses pod IPs directly, with no NAT and high performance



HOW ROUTES WORK

- External Client -> Ingress Controller -> Route -> Service -> Pods
- **Route types:** Edge (TLS terminates at ingress, most common) and Re-encrypt (re-encrypts to the service)



WHEN TO CREATE A ROUTE

- Public API endpoints, test endpoints, admin UIs, API gateway integration points
- Do not create external Routes for services that should remain internal-only



CONFIGMAPS: NON-SENSITIVE DATA

- Application configuration properties, feature flags, UI settings, log levels

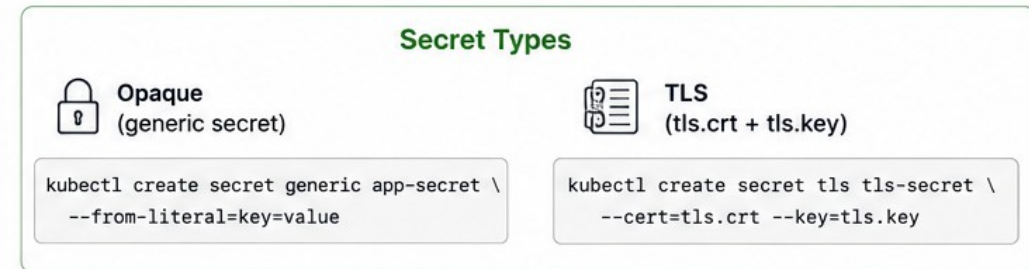
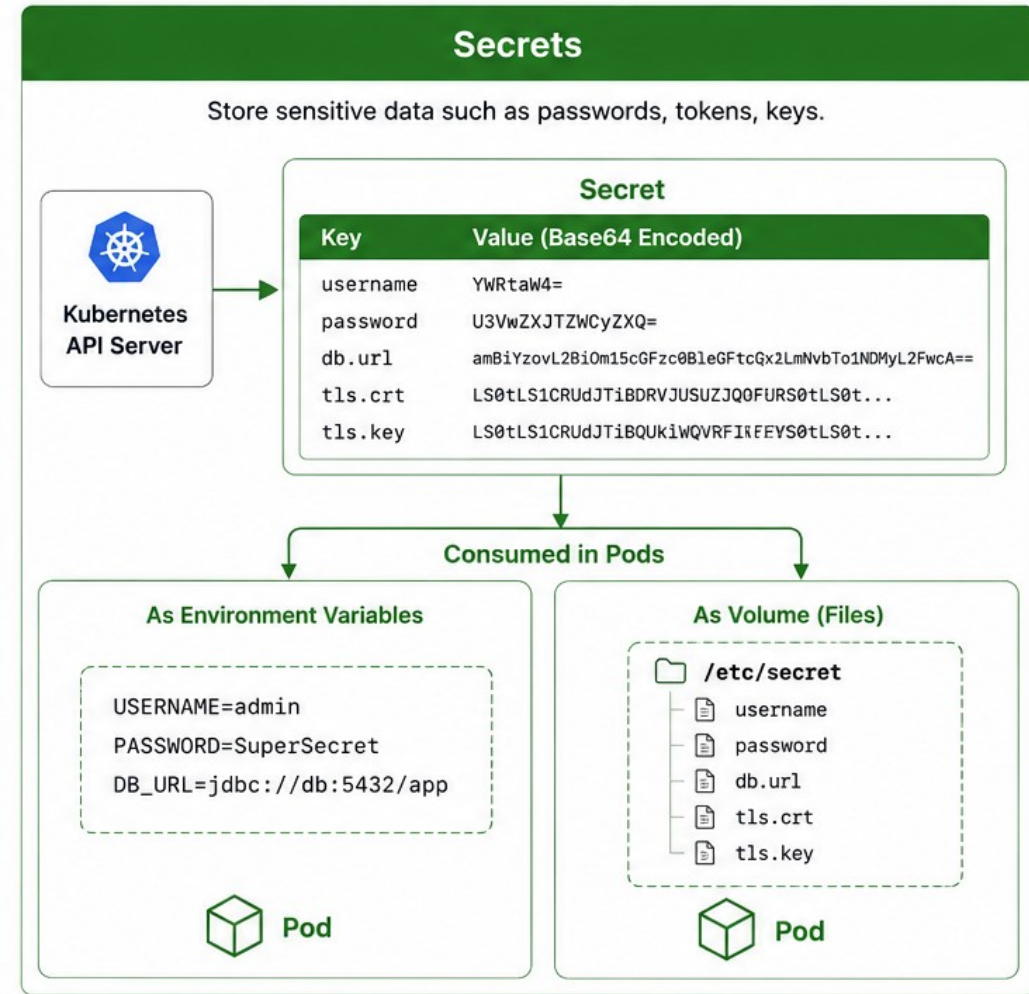
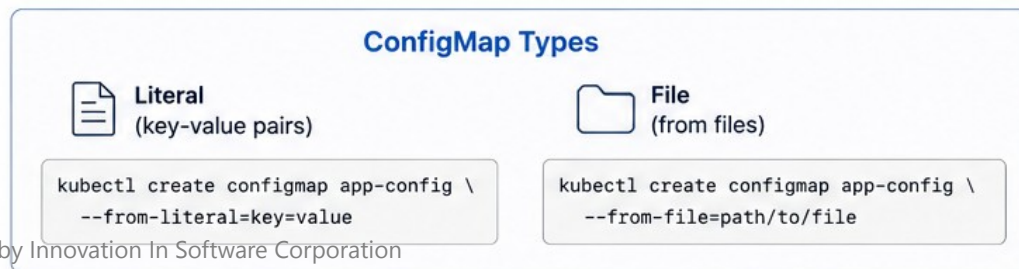
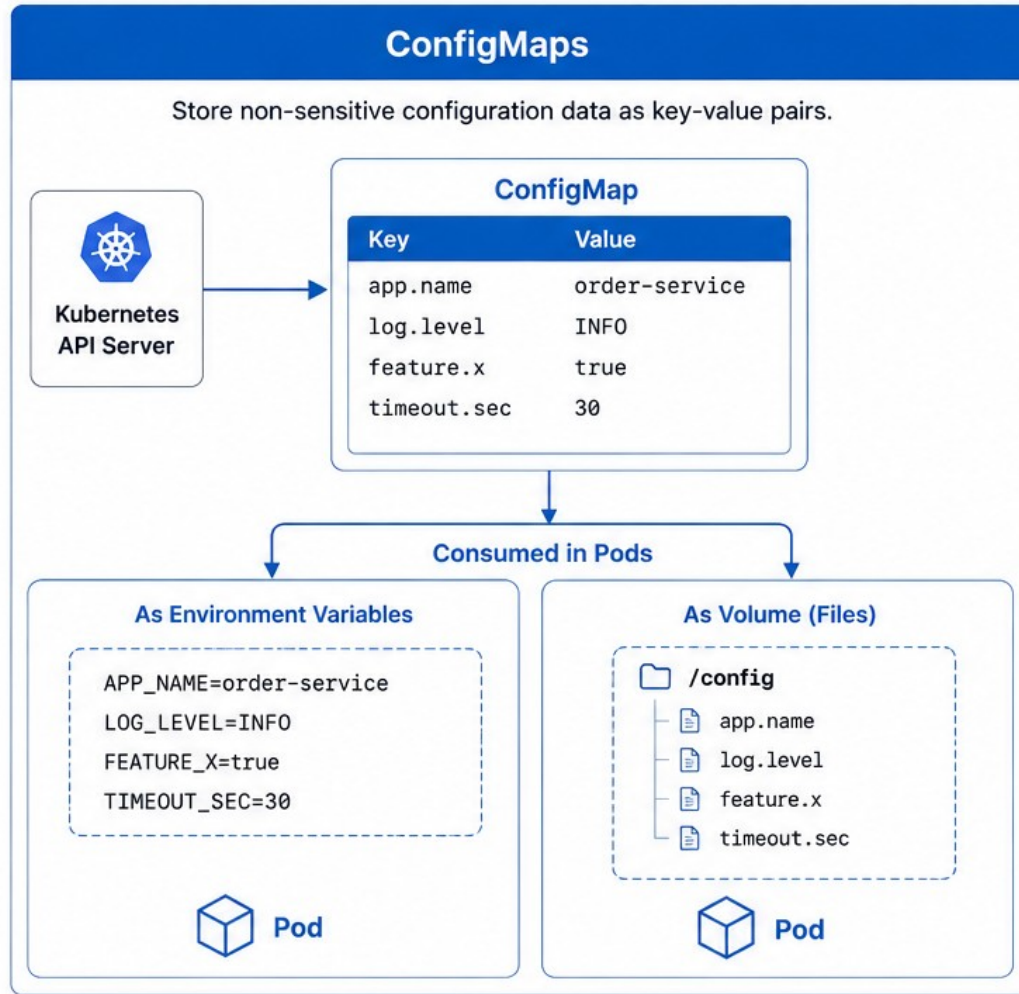


SECRETS: SENSITIVE DATA

- Database credentials, API keys/tokens, TLS certificates and private keys, OAuth client secrets
- Never store secrets in ConfigMaps; restrict access using RBAC and rotate regularly

KEY TAKEAWAY Transaction Service calls `http://account-service:8081`, not a pod IP -- OpenShift load-balances that call across every healthy Account Service pod. • Edge, re-encrypt, and passthrough TLS termination each trade off where encryption happens between the client and the pod. • Both can be mounted as files or injected as environment variables -- the only difference is that Secrets are base64-encoded and access-restricted.

ConfigMaps and Secrets



PERSISTENT VOLUMES AND CLAIMS · SERVICE ACCOUNTS AND ROLE-BASED ACCESS CONTROL



KEY CONCEPTS

- **Persistent Volume (PV):** a piece of cluster storage, provisioned by an admin or dynamically
- **Persistent Volume Claim (PVC):** a request for storage with a specific size, access mode, and storage class
- StorageClass defines the type of storage and how new volumes are provisioned



ACCESS MODES AND RECLAIM POLICIES

- RWO (ReadWriteOnce), ROX (ReadOnlyMany), RWX (ReadWriteMany)
- Retain (keep data), Delete (delete PV and storage), Recycle (deprecated)



THE FOUR-STEP RBAC MODEL

- 1. Identity:** a Service Account identifies a process running in a pod
- 2. RBAC objects:** a Role (namespaced) or ClusterRole (cluster-scoped) defines allowed actions
- 3. Binding:** a RoleBinding or ClusterRoleBinding connects the identity to the role
- 4. Access:** the bound role grants access to specific resources (pods, secrets, configmaps)

KEY TAKEAWAY RWO, ROX, and RWX access modes plus Retain/Delete reclaim policies decide who can mount a volume and what happens when a claim is deleted. • Roles define permissions, RoleBindings connect a service account to a role, and the principle of least privilege ties it all together.

OPENSIFT IMAGE STREAMS · BUILDS AND BUILDCONFIGS · OPENSIFT WEB CONSOLE AND CLI



HOW IMAGE STREAMS WORK

- An image stream is a named collection of images pulled from a source registry
- Image stream tags point to specific images (e.g., latest, v1.2.0)
- Tags can be set to automatically track updates in the source registry



BUILD STRATEGIES

- **Source-to-Image (S2I):** uses a builder image to assemble the app -- no Dockerfile needed
- **Docker strategy:** uses a Dockerfile for full control over build steps
- **Custom strategy:** runs a custom build script inside the build pod



TRIGGERS

- Builds can start manually, on a Git webhook, on image change, or on a schedule



WEB CONSOLE

- Browser-based UI to create, view, and manage projects, applications, and resources
- Monitor health, logs, metrics, and events with role-based access built in



OC CLI

- Command-line interface for full control and automation
- Ideal for scripting, CI/CD, and bulk actions across a cluster

KEY TAKEAWAY Applications reference images by image-stream name instead of a full registry path, decoupling deployments from registry locations.

- Source-to-Image (S2I) turns Java source straight into a runnable image without a Dockerfile, using a builder image as the template.
- Use the console for exploration and troubleshooting; use oc for scripting, CI/CD, and any task that needs to be repeatable.

WORKING WITH THE `oc` COMMAND



The `oc` (OpenShift CLI) is the command-line tool to interact with OpenShift clusters. It lets you manage projects, applications, and resources efficiently from your terminal.

GETTING STARTED

- **Install oc CLI**
Download from the OpenShift CLI downloads page.
- **Login to a Cluster**
`oc login <api-server>`
Authenticate and download cluster access details.
- **Set Current Context**
`oc project <project-name>`
Switch to the desired project/namespace.
- **View Current Context**
`oc whoami`
`oc project`

COMMON oc COMMANDS

TASK	COMMAND	DESCRIPTION
 List projects	<code>oc get projects</code>	List all projects you have access to
 Switch project	<code>oc project <name></code>	Set the current project
 View current user	<code>oc whoami</code>	Show the logged-in user
 List pods	<code>oc get pods</code>	List pods in the current project
 List services	<code>oc get svc</code>	List services in the current project
 List deployments	<code>oc get deploy</code>	List deployments in the current project
 Describe a resource	<code>oc describe <resource> <name></code>	Show details of a resource
 Apply configuration	<code>oc apply -f <file.yaml></code>	Create or update resources from YAML
 Delete a resource	<code>oc delete <resource> <name></code>	Delete a resource
 View logs	<code>oc logs <pod-name></code>	View logs of a pod
 Execute command in pod	<code>oc exec -it <pod-name> -- /bin/bash</code>	Open a shell in a running container

EXAMPLE WORKFLOW

```
$ oc login https://api.cluster.example.com:6443
Logged in as developer.
You have access to the following projects:
  default, my-app, dev, test, prod

$ oc project my-app
Now using project "my-app" on server "https://api.cluster.example.com:6443".

$ oc get pods
NAME                                READY   STATUS    RESTARTS   AGE
my-app-76d5f7c8c9-abc12             1/1     Running   0           2m
api-service-6f8d4fbc7d-def34        1/1     Running   0           3m
worker-5c9d8f6b7c-ghi56             1/1     Running   0           5m

$ oc get svc
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
my-app    ClusterIP   10.128.2.45   <none>        8080/TCP   2m
api-service ClusterIP   10.128.5.67   <none>        8080/TCP   3m

$ oc describe pod my-app-76d5f7c8c9-abc12 | less

$ oc logs my-app-76d5f7c8c9-abc12 -f

$ oc exec -it my-app-76d5f7c8c9-abc12 -- /bin/bash
sh-4.4$
```

- **USEFUL COMMAND GROUPS**
- `oc get` # List resources
- `oc describe` # Show details
- `oc create` # Create resources
- `oc apply` # Apply configuration
- `oc delete` # Delete resources
- `oc logs` # View logs
- `oc exec` # Execute commands
- `oc port-forward` # Forward local ports

- **QUICK TIPS**
- Use `Tab` completion: Type `'oc <TAB>'` to explore commands.
 - Use `-n` (or `--namespace`) to specify a project without switching.
`oc get pods -n dev`
 - Use `-o wide` for more details.
`oc get pods -o wide`
 - Use `oc explain <resource>` to understand resource options.
 - Use `oc help <command>` for detailed help.

- **BEST PRACTICES**
- ✓ Use service accounts and least-privilege RBAC permissions.
 - ✓ Avoid using admin credentials for day-to-day tasks.
 - ✓ Use projects to separate environments and applications.
 - ✓ Store kubeconfig securely and do not share it.
 - ✓ Regularly update the `oc` CLI to get the latest features.
 - ✓ Use `oc` in scripts and CI/CD pipelines for automation.

DECLARATIVE RESOURCE MANIFESTS · DEPLOYING SPRING BOOT TO OPENSIFT



KEY PRINCIPLES

- Declare, don't imperatively manage -- describe desired state, let the platform enforce it
- **Consistent, repeatable:** manifests are version-controlled, reviewable, and reusable
- **Idempotent:** applying the same manifest multiple times results in the same state



APPLY AND MANAGE

- `oc apply -f my-app.yaml` to create or update; `oc diff -f` to preview changes first



THE DEPLOYMENT FLOW

- 1. Prepare app:** externalise configuration, add health probes, and produce a runnable JAR
- 2. Build image:** layer the JAR into a container that runs as a non-root user
- 3. Push image:** publish to a registry with an immutable, versioned tag
- 4. Deploy:** create the Deployment so OpenShift schedules and supervises the pods
- 5. Expose:** add a Service for in-cluster traffic and a Route for external access
- 6. Access:** verify through the Route, then confirm probes and logs look healthy
 - OpenShift handles scaling, health checks, rolling updates, and high availability automatically

KEY TAKEAWAY Applying the same manifest twice produces the same result every time -- that idempotency is what makes GitOps and CI/CD pipelines trustworthy. • Source-to-Image, a Docker image, or a BuildConfig are three different paths to the same destination: a running, exposed Spring Boot service.

CONFIGURING RUNTIME ENVIRONMENTS · RESOURCE REQUESTS AND LIMITS



THREE CONFIGURATION SOURCES

- **ConfigMaps:** non-sensitive configuration
- **Secrets:** sensitive data (encrypted)
- **Environment variables:** runtime overrides and simple values



BEST PRACTICES

- Externalize all configuration -- no hard-coded values
- Validate configuration on startup and fail fast when invalid



QUALITY OF SERVICE CLASSES

- **Guaranteed:** requests == limits for CPU and memory -- highest priority, least likely to be evicted
- **Burstable:** requests < limits -- medium priority
- **BestEffort:** no requests or limits set -- lowest priority, most likely to be evicted

KEY TAKEAWAY Same image, different configuration -- the twelve-factor principle that keeps a Spring Boot service portable across every environment. • Guaranteed, Burstable, and BestEffort quality-of-service classes decide which pods get evicted first when a node runs out of resources.

LIVENESS, READINESS, AND STARTUP PROBES · HORIZONTAL POD AUTOSCALING · ROLLING DEPLOYMENTS



THE THREE PROBE TYPES

- **Liveness:** is the container running? Failure restarts it
- **Readiness:** is it ready for traffic? Failure removes it from service endpoints
- **Startup:** has the app finished starting? Disables liveness/readiness until it succeeds



SPRING BOOT EXAMPLE ENDPOINTS

- /actuator/health/liveness and /actuator/health/readiness



HOW HPA WORKS

- Metrics Server collects CPU/memory/custom metrics from pods
- HPA Controller compares metrics to target values and decides the desired replica count
- The Deployment is adjusted to match -- more pods handle more traffic, fewer save resources



HOW A ROLLING UPDATE WORKS

- A new pod with the new version is created; an old pod is terminated once the new one is ready
- The process repeats until all old pods are replaced -- traffic is always served



ROLLBACK

- `oc rollout undo deployment/<name>` reverts to the previous revision with one command

KEY TAKEAWAY A startup probe protects a slow-starting Spring Boot app from being killed by liveness checks before it's even finished initializing. • `oc scale` changes replicas manually; a `HorizontalPodAutoscaler` does the same thing continuously, in response to real load. • `maxUnavailable` and `maxSurge` control exactly how aggressively the rollout proceeds, balancing speed against risk.

BLUE-GREEN DEPLOYMENTS · CANARY DEPLOYMENTS · ROLLBACKS AND DEPLOYMENT HISTORY



HOW BLUE-GREEN WORKS

- Deploy the new version (Green) alongside the current one (Blue), fully in parallel
- Validate Green thoroughly, then switch traffic from Blue to Green instantly
- Retire Blue once Green proves stable -- or revert instantly if it doesn't



HOW ROLLBACKS WORK

- A new version is deployed; monitoring detects errors or performance problems
- Rollback is triggered via CLI, web console, or API; traffic routes back to the last stable version



HOW CANARY DEPLOYMENT WORKS

- Deploy the new version alongside the stable one, routing only a small percentage of traffic to it
- Monitor metrics, logs, and user feedback; gradually increase traffic if healthy
- Automatic or manual rollback if errors or performance issues are detected



USEFUL COMMANDS

- `oc rollout history deployment/<name>` to view revisions
- `oc rollout undo deployment/<name> --to-revision=<n>` to roll back to a specific one

KEY TAKEAWAY Green gets fully validated with real traffic patterns before the switch, and Blue stays on standby as an immediate rollback target.

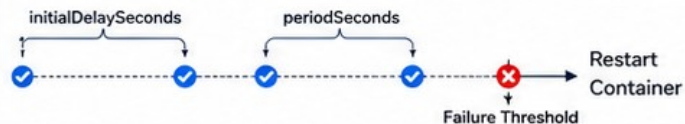
- 5% traffic to the canary, monitor error rate and latency, then gradually increase to 25%, 50%, and 100% -- or roll back at any step.
- `oc rollout undo deployment/<name>` reverts to the last stable revision in seconds, without rebuilding a single container.

Probes



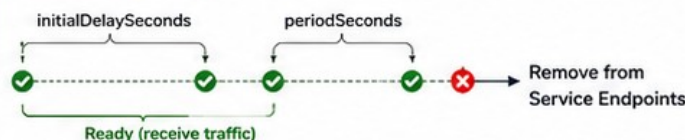
Liveness Probe

Detects if container is alive.



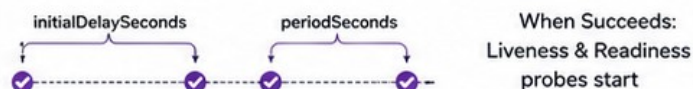
Readiness Probe

Determines if container is ready to serve traffic.



Startup Probe

Detects if application has started.



Resources

Requests (Minimum guaranteed)



CPU

request: 250m
(0.25 vCPU)



Memory

request: 256Mi

Limits (Maximum allowed)



CPU

limit: 1
(1 vCPU)



Memory

limit: 512Mi

If a container exceeds its limit:



CPU Throttled



Memory OOMKilled

Scaling

Horizontal Pod Autoscaler (HPA)

Metrics
(CPU/Memory
/Custom)



HPA



Adjust Replicas
1 → N

Manual Scaling

```
$ kubectl scale deploy myapp --replicas=5
```

Vertical Pod Autoscaler (VPA)

Observe
Resource Usage



VPA



Recommend/
Update Requests
and Limits



Note: VPA works with appropriate update modes.

Rollout

1. Current State

ReplicaSet: v1
(3 pods)

2. Rolling Update

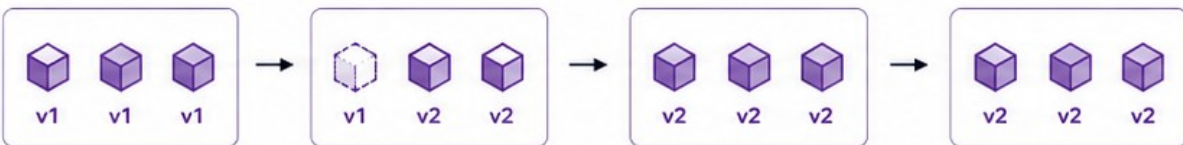
New ReplicaSet: v2
created

3. Rolling Update

Replacing old pods
gradually

4. Update Complete

All pods running new
version



Zero Downtime

Useful Commands

```
kubectl set image deploy/myapp app=v2 | kubectl rollout status deploy/myapp | kubectl rollout history deploy/myapp
```

Rollback

1. Current State

(v2 running)

2. Rollback Initiated

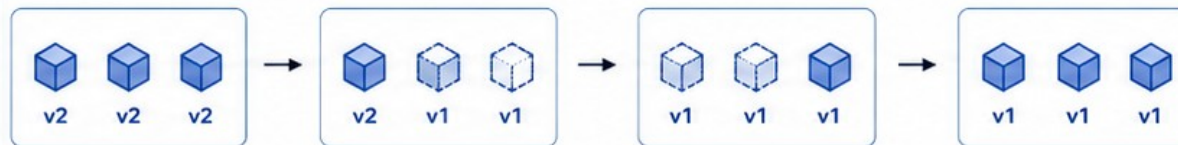
Rolling back to
previous version (v1)

3. Rolling Back

Replacing v2 pods
with v1

4. Rollback Complete

All pods running
previous version (v1)



Zero Downtime

Useful Commands

```
kubectl rollout undo deploy/myapp | kubectl rollout undo deploy/myapp --to-revision=2 | kubectl rollout history deploy/myapp
```

OPENSIFT LOGGING AND MONITORING · TROUBLESHOOTING PODS AND DEPLOYMENTS

COMMON POD ISSUES

ISSUE	WHAT TO CHECK
Pod Pending	oc describe pod, oc get events
CrashLoopBackOff	oc logs --previous, oc describe pod
Readiness/Liveness failure	oc describe pod, check probe path/port
Volume/Storage issue	oc describe pod, oc get pvc

WHAT GETS COLLECTED

- Application logs, container/pod logs, cluster and node metrics
- API server and audit logs, events and alerts

COMMON TOOLS

- Prometheus (metrics), Grafana (dashboards), Elasticsearch (log storage/search), Alertmanager (alert routing)

KEY TAKEAWAY Fluentd/Fluent Bit ship logs to Elasticsearch, Prometheus scrapes metrics, and Grafana/Kibana turn both into dashboards and alerts. • oc describe pod gives the high-level view first; events almost always contain the most accurate clue about what actually failed.

OPENSIFT SECURITY CONTEXT CONSTRAINTS



Control privilege and security settings for Pods at the cluster level

OVERVIEW

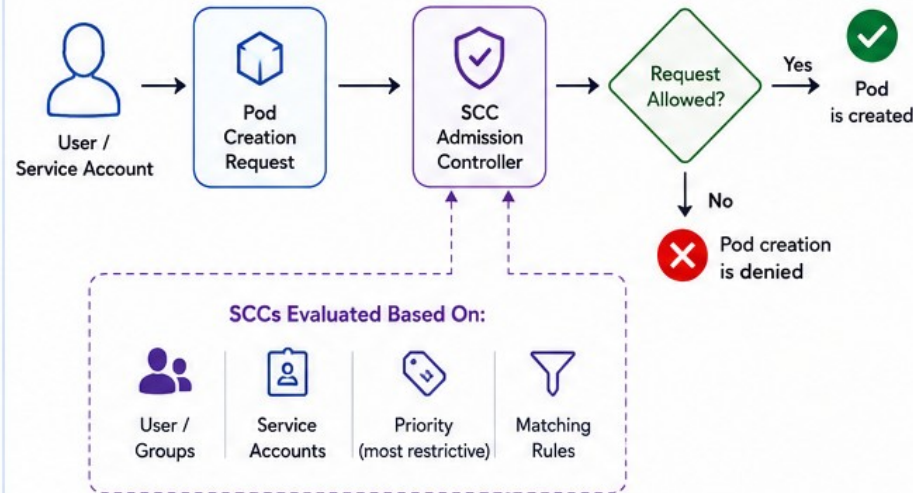
Security Context Constraints (SCCs) define the set of security privileges and conditions that a Pod or Container must run with.

SCCs are cluster-level resources used by OpenShift to enforce multi-tenancy security and prevent containers from gaining root or escalated privileges.

KEY CONCEPTS

- SCCs restrict what a user/service account can run.
- They control privileges such as UID, GID, capabilities, host access, volumes, SELinux, and more.
- At pod creation, OpenShift validates the Pod's security context against the SCCs the user/service account can use.
- The most restrictive matching SCC is applied.

HOW SCCs WORK



WHAT SCCs CONTROL



COMMON BUILT-IN SCCs

SCC NAME	PURPOSE
restricted (default)	Most restrictive. Non-root, minimal privileges.
nonroot	Must run as non-root UID.
anyuid	Allows any UID (including root).
privileged	Full privileges. Similar to root on the host.
hostnetwork	Allows use of host network namespace.
hostaccess	Allows access to host namespaces and devices.
nonroot-v2	Enhanced non-root with additional restrictions.

Use built-in SCCs when possible.
Avoid using privileged or anyuid unless absolutely required.

USEFUL OC COMMANDS

<code>oc get scc</code>	List all SCCs
<code>oc describe scc <scc-name></code>	View SCC details
<code>oc get scc -o wide</code>	View SCCs with more info
<code>oc adm policy who-can use scc <scc-name></code>	See who can use an SCC
<code>oc adm policy add-scc-to-user <scc> <user></code>	Grant SCC to user
<code>oc adm policy add-scc-to-group <scc> <group></code>	Grant SCC to group
<code>oc adm policy add-scc-to-serviceaccount <scc> <ns>:<sa></code>	Grant SCC to service account
<code>oc adm policy remove-scc-from-user <scc> <user></code>	Revoke SCC from user

BEST PRACTICES

Follow least privilege.
Grant only what is required.

Use restricted or nonroot SCC by default.

Avoid anyuid and privileged SCCs in production.

Review and audit SCC usage regularly.

Test workloads with correct SCC before deployment.

Keep OpenShift and SCC definitions up to date.

OPENSIFT NETWORKING AND NETWORK POLICIES · MANAGING APPLICATION SECRETS

TYPES OF SECRETS

TYPE	EXAMPLE USE
Opaque (generic)	Passwords, tokens, API keys
kubernetes.io/dockerconfigjson	Registry authentication / image pull secrets
kubernetes.io/tls	TLS termination, mTLS
Service Account Token	Pod authentication to the API server

NETWORK POLICY BASICS

- Network Policies define rules for controlling ingress and egress traffic to/from pods
- Applied per namespace and pod selector; if no policy selects a pod, all traffic is allowed
- Ingress rules answer 'who can talk to me?'; egress rules answer 'where can I talk?'

KEY TAKEAWAY Once a NetworkPolicy selects a pod, only traffic matching its rules is allowed -- everything else is silently denied. • Opaque, docker registry, TLS, and service-account-token secrets each cover a different sensitive-data use case in OpenShift.

OPENSIFT PRODUCTION-READINESS CHECKLIST



Key areas to validate before promoting applications to production

PURPOSE

Use this checklist to ensure your OpenShift applications are secure, resilient, observable, and ready for production workloads.

HOW TO USE

- ✓ Review each category and ensure all items are completed.
- ✓ Work with your team to address any gaps.
- ✓ Re-run the checklist before every production release.

LEGEND

- ✓ Complete
- In Progress
- ✗ Not Started
- N/A

1. APPLICATION DEPLOYMENT

- ☐ Images are immutable and from a trusted registry
- ☐ Deployment uses resource requests and limits
- ☐ Liveness, readiness, and startup probes configured
- ☐ Rolling update strategy configured
- ☐ Health checks verified
- ☐ No "latest" image tags in production
- ☐ Pods start successfully and are stable

2. CONFIGURATION AND SECRETS

- ☐ ConfigMaps used for non-sensitive config
- ☐ Sensitive data stored in OpenShift Secrets
- ☐ Secrets are mounted or injected securely
- ☐ No secrets in images, env files, or code
- ☐ Secret access restricted via RBAC
- ☐ External secret store integrated (if required)
- ☐ Configuration validated for all environments

3. SECURITY

- ☐ Service Accounts are scoped and least-privilege
- ☐ RBAC roles and bindings are reviewed
- ☐ SCCs are appropriate and least-privilege
- ☐ Network Policies restrict traffic as needed
- ☐ Containers run as non-root
- ☐ FSGroup and securityContext set as needed
- ☐ Image vulnerability scan completed
- ☐ TLS/mTLS enabled for in-transit encryption
- ☐ Audit logging enabled

4. NETWORKING

- ☐ Services expose only required ports
- ☐ Routes use TLS and secure certificates
- ☐ Network Policies defined and applied
- ☐ Egress traffic is restricted appropriately
- ☐ DNS resolution inside the cluster verified
- ☐ No exposed NodePorts unless required

5. STORAGE AND PERSISTENCE

- ☐ PersistentVolumeClaims defined with correct size
- ☐ StorageClass with suitable replication/HA
- ☐ Backups and snapshots configured (if required)
- ☐ ReadWriteOnce/Many policies validated
- ☐ Data retention and recovery plan in place

6. OBSERVABILITY

- ☐ Centralized logging enabled (Cluster Logging)
- ☐ Metrics collected (Prometheus/Micrometer)
- ☐ Dashboards and alerts configured (Grafana)
- ☐ Distributed tracing enabled (OpenTelemetry)
- ☐ Alerts tested and routed
- ☐ Log retention policies configured

7. RESILIENCE AND SCALABILITY

- ☐ Resource limits prevent noisy neighbors
- ☐ Horizontal Pod Autoscaler configured (if needed)
- ☐ PDBs (Pod Disruption Budgets) configured
- ☐ Anti-affinity rules configured where required
- ☐ Multi-replica deployment verified
- ☐ Rolling update and rollback tested

8. CI/CD AND RELEASE

- ☐ CI/CD pipeline defined as code
- ☐ Automated tests and quality gates in place
- ☐ Image scanning and policy checks enabled
- ☐ Approval process for production promotion
- ☐ Rollback strategy validated
- ☐ Release notes and versioning maintained

9. OPERATIONS

- ☐ Runbooks and SOPs documented
- ☐ Backup, restore, and DR plan documented
- ☐ Capacity and performance tested
- ☐ Access and monitoring reviewed
- ☐ On-call and escalation processes defined
- ☐ Production smoke tests completed



SUCCESS CRITERIA

All critical items are complete.
No high-priority gaps remain.
Team sign-off obtained.



COMMON RISKS

- Overly permissive RBAC or SCCs
- Missing resource limits or probes
- Unsecured secrets or open routes



TIP

Automate as much as possible.
Continuously validate in lower environments before production.

APPROVED FOR PRODUCTION

Team: _____
Date: _____
Reviewer: _____

EXERCISE 4.2 — CONTAINERIZE ACCOUNT SERVICE

Module 10 checkpoint B — Secure non-root image.



OBJECTIVE

- Secure non-root image
- 10 min · ops warmup checkpoint
- **Worksheet:**
curriculum/day-04/exercises/exercise-4.2-containerize-account.md



DO THIS

- 1. Multi-stage build:** Maven compile in builder, JRE runtime in final image
- 2.** USER md287 (non-root). OpenShift also sets runAsNonRoot: true
- 3.** Immutable tag 1.0.0 — not :latest
- 4.** Images are built from Lab 3 solution context — do not recopy Java trees



EXPECTED RESULT

- You can explain why the runtime user is not root before you run docker build.
- **Lab 4 Step 3 recap:** multi-stage Containerfile, USER md287, tag md287/account-service:1.0.0.

KEY TAKEAWAY You can explain why the runtime user is not root before you run docker build.